

Web Application Vulnerability Scanner

Abstract

With the rapid growth of web-based applications, security vulnerabilities have become a major concern. This project focuses on the development of a Python-based Web Application Vulnerability Scanner designed to identify common security flaws such as Cross-Site Scripting (XSS), SQL Injection (SQLi), and Cross-Site Request Forgery (CSRF). The scanner automates the process of crawling web pages, detecting input fields, injecting malicious payloads, and analyzing server responses. A user-friendly web interface built using Flask allows users to initiate scans and view detailed results. The tool aims to assist developers and security professionals in identifying and mitigating vulnerabilities effectively.

Introduction

Web applications are frequently targeted by attackers exploiting common vulnerabilities listed in the OWASP Top 10. Manual vulnerability testing is time-consuming and requires expertise, making automated tools essential in modern cybersecurity practices.

This project presents a lightweight and efficient Web Application Vulnerability Scanner developed using Python. The scanner is capable of detecting critical vulnerabilities such as XSS, SQL Injection, and CSRF by analyzing user inputs and server responses. It provides an easy-to-use interface and generates structured reports, making it suitable for beginners as well as professionals in ethical hacking and penetration testing.

Tools Used

The following tools and technologies were used to develop the scanner:

- **Python** – Core programming language for building the scanner logic
- **requests** – To send HTTP requests and interact with web servers
- **BeautifulSoup** – For parsing HTML content and extracting forms, inputs, and links
- **Flask** – To develop a web-based user interface for controlling scans
- **Regex (Regular Expressions)** – For detecting vulnerability patterns in responses
- **OWASP Top 10 Checklist** – As a reference for identifying common vulnerabilities

Steps Involved in Building the Project

1. Requirement Analysis

Identified the need to detect common web vulnerabilities such as XSS, SQL Injection, and CSRF.

2. **Web Crawling Implementation**

Used Python's requests library to fetch web pages and BeautifulSoup to extract links, forms, and input fields.

3. **Input Field Detection**

Parsed HTML to identify all possible input points such as text fields, search boxes, and form submissions.

4. **Payload Injection**

Designed and injected test payloads (e.g., `<script>alert(1)</script>` for XSS, `' OR 1=1 --` for SQLi) into input fields.

5. **Response Analysis**

Analyzed server responses using pattern matching and regex to detect vulnerability indicators.

6. **CSRF Detection**

Checked for missing or weak CSRF tokens in forms to identify potential CSRF vulnerabilities.

7. **Logging and Reporting**

Logged detected vulnerabilities with details such as URL, payload used, evidence, and severity level.

8. **Flask Web Interface Development**

Created a simple UI where users can enter target URLs, initiate scans, and view results in a structured format.

9. **Testing and Validation**

Tested the scanner on intentionally vulnerable applications to verify accuracy and effectiveness.

Conclusion

The Web Application Vulnerability Scanner successfully demonstrates how automation can simplify the process of identifying common security flaws in web applications. By integrating web crawling, payload injection, and response analysis, the tool provides an efficient method for vulnerability detection. The addition of a Flask-based user interface enhances usability and accessibility.

This project serves as a foundational step toward building advanced security tools. Future enhancements may include support for additional vulnerabilities, improved detection accuracy, multi-threaded scanning, and detailed PDF/HTML report generation. Overall, the scanner contributes to improving web application security and supports learning in ethical hacking and penetration testing.